OWASP Juice Shop VAPT Report

Prepared by: Ahammed Zidan M

Test Date: 27 June 2026

Target: OWASP Juice Shop running locally at http://localhost:3001

Assessment Type: Web Application Vulnerability Assessment and Penetration Testing

Executive Summary

This assessment was performed against OWASP Juice Shop, an intentionally vulnerable web application deployed locally in a Docker container. The goal was to practice a realistic VAPT workflow: reconnaissance, automated scanning, manual verification, exploitation of selected vulnerabilities, evidence collection, and professional reporting.

The assessment identified multiple vulnerabilities across authentication, information disclosure, directory exposure, error handling, and security headers. The most severe finding was a SQL injection authentication bypass that allowed administrative login without valid credentials.

Scope

In scope: local OWASP Juice Shop instance at http://localhost:3001, web application paths, and public application functionality.

Out of scope: third-party production systems, denial-of-service testing, social engineering, Docker host attacks, and external networks.

Tools Used

Docker, Nmap, Nikto, browser-based manual testing, and OWASP Juice Shop.

Findings Summary

ID	Finding	Severity
F-01	SQL Injection Authentication Bypass	Critical
F-02	Confidential Document Exposure	High
F-03	Exposed FTP Directory Listing	Medium
F-04	Verbose Error / Stack Trace Disclosure	Medium
F-05	Missing Security Headers	Medium
F-06	Overly Permissive CORS Header	Medium
F-07	Robots.txt Sensitive Path Disclosure	Low

Detailed Findings

F-01: SQL Injection Authentication Bypass

Severity	Critical
Affected	Login page

Description

The login function was vulnerable to SQL injection. By entering a crafted SQL payload into the email field and any value in the password field, authentication was bypassed and administrative access was obtained.

Evidence

Payload used: ' OR 1=1--. The application displayed a success message that the Login Admin challenge was solved. Evidence: screenshots/09-sqli-admin-login-bypass.png and screenshots/10-admin-session-evidence.png.

Impact

An attacker could bypass authentication and gain unauthorized administrative access without valid credentials.

Recommendation

Use parameterized queries or prepared statements, validate and sanitize input, avoid string concatenation in database queries, and monitor suspicious login attempts.

F-02: Confidential Document Exposure

Severity	High
Affected	http://localhost:3001/ftp/acquisitions.md

Description

A confidential internal document was publicly accessible through the exposed /ftp/ directory.

Evidence

The file acquisitions.md contained the statement 'This document is confidential! Do not distribute!' and described planned acquisitions. Evidence: screenshots/08-confidential-document-exposure.png.

Impact

Unauthorized access to confidential business documents could lead to information disclosure, reputational damage, insider trading risks, and competitive intelligence exposure.

Recommendation

Remove confidential documents from public directories, enforce access controls, disable directory listing, and implement secure file storage.

F-03: Exposed FTP Directory Listing

Severity	Medium

Affected	http://localhost:3001/ftp/

Description

The application exposed a publicly accessible directory listing at /ftp/ containing backup files, markdown documents, package files, and an encrypted database file.

Evidence

Visible files included acquisitions.md, coupons_2013.md.bak, incident-support.kdbx, package.json.bak, and suspicious_errors.yml. Evidence: screenshots/05-ftp-directory-listing.png.

Impact

An attacker could gather application information, identify backup files, analyze package data, or retrieve sensitive documents.

Recommendation

Disable directory listing, restrict sensitive file paths, remove backup/configuration files from the web root, and enforce proper access controls.

F-04: Verbose Error / Stack Trace Disclosure

Severity	Medium
Affected	http://localhost:3001/ftp/package.json.bak

Description

A restricted backup file request returned a verbose 403 error containing internal framework details, Express version information, file paths, and stack trace data.

Evidence

The response disclosed OWASP Juice Shop (Express ^4.22.1), /juice-shop/build/routes/fileServer.js, and /juice-shop/node_modules/express/. Evidence: screenshots/07-verbose-error-disclosure.png.

Impact

Verbose errors can help attackers fingerprint the technology stack and understand internal routing logic.

Recommendation

Disable verbose errors in production, return generic error pages, avoid exposing framework versions and internal paths, and log detailed errors server-side only.

F-05: Missing Security Headers

Severity	Medium
Affected	http://localhost:3001/

Description

Nikto identified missing recommended security headers including Content-Security-Policy, Referrer-Policy, Permissions-Policy, Strict-Transport-Security, and X-Content-Type-Options.

Evidence

Evidence: screenshots/04-nikto-scan-result.png.

Impact

Missing security headers can increase exposure to client-side attacks such as XSS, data leakage through referrers, MIME sniffing, and unsafe browser feature access.

Recommendation

Implement appropriate HTTP security headers, tuned to the application context.

F-06: Overly Permissive CORS Header

Severity	Medium
Affected	http://localhost:3001/

Description

The application returned Access-Control-Allow-Origin: *, allowing any origin.

Evidence

Nikto detected the wildcard CORS header. Evidence: screenshots/04-nikto-scan-result.png.

Impact

If combined with sensitive endpoints or weak authentication controls, permissive CORS can increase cross-origin data exposure risk.

Recommendation

Restrict CORS to explicitly trusted origins and avoid wildcard origins for sensitive applications.

F-07: Robots.txt Sensitive Path Disclosure

Severity	Low
Affected	http://localhost:3001/robots.txt

Description

The robots.txt file disclosed the /ftp directory.

Evidence

The file contained User-agent: * and Disallow: /ftp. Evidence: screenshots/06-robots-txt-discloses-ftp.png.

Impact

Robots.txt is public and should not be used as access control. Disclosed paths can help attackers discover hidden directories.

Recommendation

Do not rely on robots.txt to protect sensitive paths. Enforce access controls server-side.

Conclusion

This project demonstrated a practical web application security testing workflow from reconnaissance through exploitation and reporting. The assessment reinforced secure query handling, access control, safe file storage, generic error handling, secure HTTP headers, and manual validation of scanner findings.